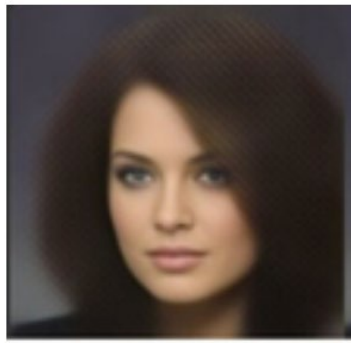
Evaluation of Generative Models

DSAIT4030 2025/26

Zonghuan Li
PhD Student
Department of Intelligent Systems, TU Delft

How to evaluate generative models?

- Which model is better?



Model A



Model B

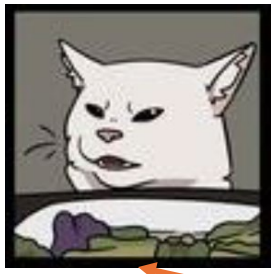
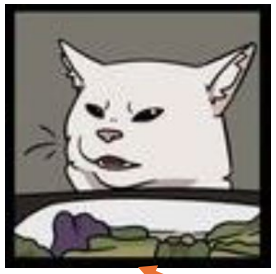
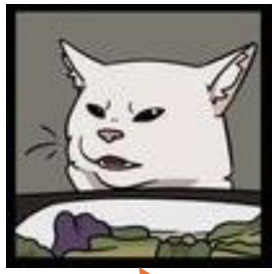


Model C

- What do we have at hand?
  - Test set data
  - Generate samples
  - (Sometimes) density function

Sources: *https://gaussian37.github.io/deep-learning-chollet-8-4/* *https://yang-song.net/blog/2021/score/* *https://www.mdpi.com/1099-4300/26/6/479*

How to evaluate generative models?

- Which model is better?
- $p_A($  $),$ $p_B($  $),$ $p_C($  $)$

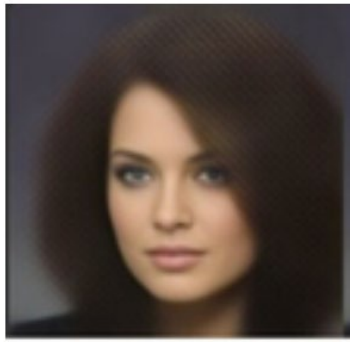
- Density evaluation
  - **Which probability is higher for test data?**



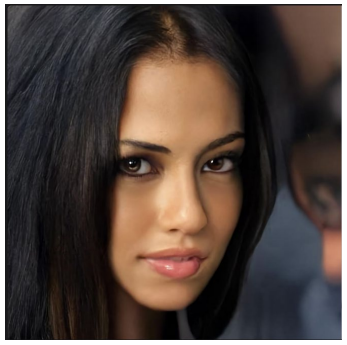
Test data

Cat images from RedBubble

How to evaluate generative models?



Model A



Model B



Model C



Sample evaluation:
which model generates data that is close enough to test data distribution?

Evaluation paradigm

- Usually we have train set and test set
- After training on the train set, we apply the model to test set
  - Density evaluation
    - Are the models assigning test data high probability?
  - Sample evaluation
    - Are samples generated from the model good enough? (Or are they “close” enough to the distribution of test set data?)
  - Downstream tasks
    - Is the model supporting desired application well?

- **Evaluating GMs is multi-objective**

Learning goals

- Evaluation by likelihood
  - Direct computation or approximation
- Evaluation by sampling
  - IS
  - FID
  - MMD & KID
  - Precision & recall
- Other concerns of evaluation
  - Recognizing overfitting
  - Drawbacks of lack of pretrained classifier

Explicit likelihood

- KL divergence between data and model distribution p(x) and q(x)

$$D_{\mathbb{KL}}(p||q) = \int p(\mathbf{x}_n) \log \frac{p(\mathbf{x}_n)}{q(\mathbf{x}_n)} = \int p(\mathbf{x}_n) \log p(\mathbf{x}_n) - \int p(\mathbf{x}_n) \log q(\mathbf{x}_n) = \boxed{-\mathbb{H}(p)} + \boxed{\mathbb{H}_{ce}(p, q)}$$

Constant

Approximated by using test data

- Cross entropy on test data becomes Negative log likelihood (NLL):

$$\mathbb{H}_{ce}(p, q) = -\frac{1}{N} \sum_{i=1}^{n} \boxed{p(\mathbf{x}_n)} \log q(\mathbf{x}_n) = -\frac{1}{N} \sum_{i=1}^{n} \log q(\mathbf{x}_n) = \text{NLL}$$

This is just 1

- Small KL means large likelihood to data from test set
- We have seen this before (lecture 3)

$$\text{KL}(p_{\text{data}}, p_\theta) = \mathbb{E}_{\mathbf{X} \sim p_{\text{data}}} \left[\log \frac{p_{\text{data}}(\mathbf{X})}{p_\theta(\mathbf{X})} \right]$$

$$= \underbrace{\mathbb{E}_{\mathbf{X} \sim p_{\text{data}}} [\log p_{\text{data}}(\mathbf{X})]}_{\text{does not depend on } \theta} - \underbrace{\mathbb{E}_{\mathbf{X} \sim p_{\text{data}}} [\log p_\theta(\mathbf{X})]}_{\text{expected log-likelihood}}$$

Minimizing KL divergence is equivalent to maximizing likelihood!

Implicit likelihood

- Not all models have tractable likelihoods
- VAE
  - Importance sampling
- Energy based models
  - Annealed importance sampling
- GAN
  - Non-parametric kernel density estimation (KDE)

Importance sampling

- What if we want to estimate the density function in VAE?
  - Suppose we have sample x
- We can use Monte Carlo

$$p_\theta(x) = \mathbb{E}_{z\sim p(z)}[p_\theta(x|z)] = \frac{1}{K}\sum_{k=1}^{K} p_\theta\left(x|z^{(k)}\right)$$

Decoder computes this

- Plain Monte Carlo samples from $p(z)$ directly
- **Think**: suppose x is a written 7. How are the values of $p_\theta(x|z)$ for most drawn z?
  - Tiny values and not very efficient for estimation

Importance sampling

- Consider another way of sampling

Importance weight

$$p_\theta(x) = \sum_{z \in \mathcal{Z}} p_\theta(x, z) = \sum_{z \in \mathcal{Z}} \frac{q(z)}{q(z)} p_\theta(x, z) = \mathbb{E}_{z \sim q(z)} \left[\frac{p_\theta(x, z)}{q(z)} \right] \approx \frac{1}{K} \sum_{k=1}^{K} \left[\frac{p_\theta(x|z^{(k)}) p(z^{(k)})}{q(z^{(k)})} \right]$$

- We can sample from another $q(z)$ instead of original $p(z)$
- Using $q(z) = p_\theta(z|x)$ gives ground truth, but it's intractable. In practice choose $q(z) \approx q_\phi(z|x)$.
- ***Sample around latent codes that are more likely to reconstruct x (more informative)***

Kernel density estimation (KDE)

- What if we can only sample from the model (for example GANs)?
  - We can estimate the density function
- Suppose we have 6 data samples drawn from the model

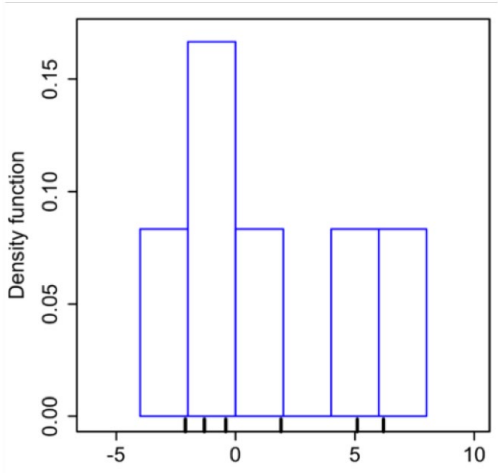
$\mathbf{x}^{(1)}$	$\mathbf{x}^{(2)}$	$\mathbf{x}^{(3)}$	$\mathbf{x}^{(4)}$	$\mathbf{x}^{(5)}$	$\mathbf{x}^{(6)}$
-2.1	-1.3	-0.4	1.9	5.1	6.2

- What is $p_\theta(-0.5)$?
- **Answer 1**: $p_\theta(-0.5) = 0$, since -0.5 is not in the samples

https://github.com/kuleshov-group/dgm-resources/blob/main/slides_pdf/lecture14.pdf

Kernel density estimation (KDE)

- **Answer 2**: we can estimate the density function by creating some bins



- Choose bin width = 2, min height = 1/12 (area of total bins equals 1)
- $p_\theta(-0.5) = 1/6, p_\theta(0.5) = 1/12, p_\theta(3) = 0$

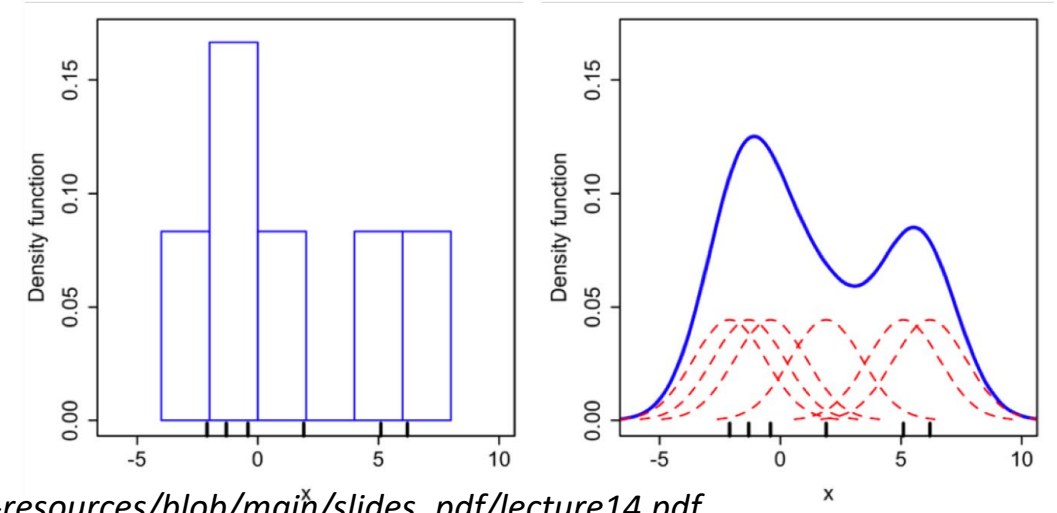
https://github.com/kuleshov-group/dgm-resources/blob/main/slides_pdf/lecture14.pdf

Kernel density estimation (KDE)

- **Answer 3**: use kernel function instead of rectangular bins

$$\hat{p}(x) = \frac{1}{n} \sum_{x^{(i)} \in \mathcal{S}} K\left(\frac{x - x^{(i)}}{\sigma}\right)$$

- Example: Gaussian kernel $K(u) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}u^2\right)$
- Similar to mixture model



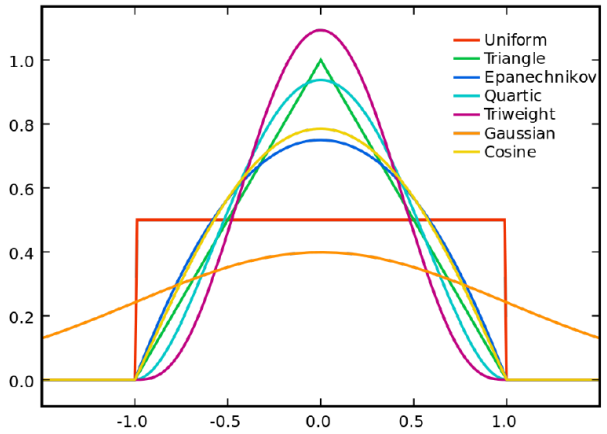
https://github.com/kuleshov-group/dgm-resources/blob/main/slides_pdf/lecture14.pdf

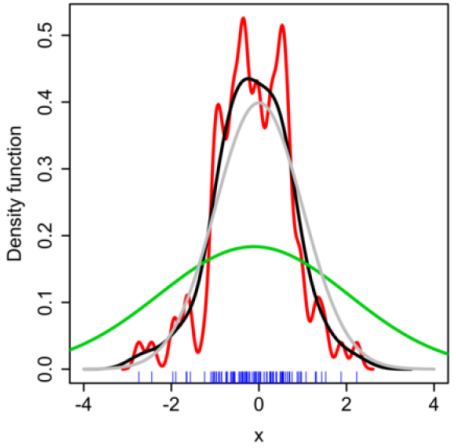
Kernel density estimation (KDE)

- Choice of kernel functions K such that $\int K(u)du = 1, \quad K(u) = K(-u)$
  - Measures similarity between points

$$\hat{p}(x) = \frac{1}{n} \sum_{x^{(i)} \in \mathcal{S}} K\left(\frac{x - x^{(i)}}{\sigma}\right)$$

Controls smoothness





Grey: true density
Black: optimal
Red: undersmooth
Green: oversmooth

- KDE is unreliable in high dimensions

https://github.com/kuleshov-group/dgm-resources/blob/main/slides_pdf/lecture14.pdf

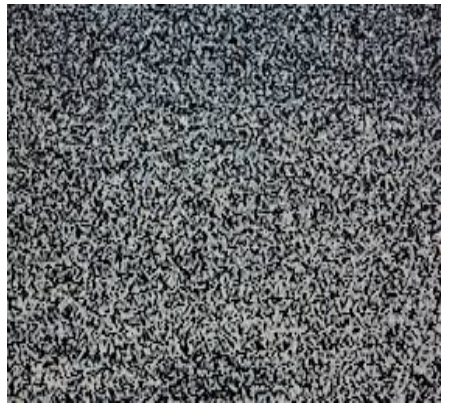
Likelihood estimation

- Do you think good likelihood always ensure good sample quality?
- … and vice versa?

- **No!**
- We’ll show soon (in theory)!

Likelihood doesn't imply sample quality

- Suppose a good model density is $p(x)$, and bad model (e.g. white noise) density is $q(x)$, with data dimension of d

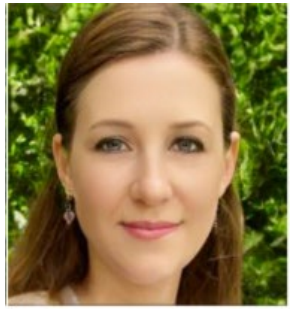
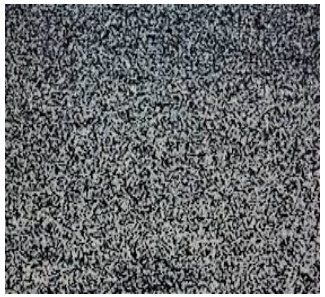
0.01  $+ 0.99$ 

$p(x)$ $q(x)$

- Consider model $0.01p(\mathbf{x}) + 0.99q(\mathbf{x})$
- Sample quality? Poor! (99% chance noise)

Likelihood doesn't imply sample quality

- How about likelihood?

0.01  $+\ 0.99$ 

$p(x)$ $q(x)$

$$\log\left[0.01p(\mathbf{x}) + 0.99q(\mathbf{x})\right] \geq \log\left[0.01p(\mathbf{x})\right] = \boxed{\log p(\mathbf{x})} - \boxed{\log 100}$$

Proportional to dimension d Constant

- With high dimension d, likelihood can be very large
  - 32*32*3 image -> $\mathrm{d} = 3072$, and $\log p(x)$ may be something like -3000
  - While $\log 100 < 5$

Likelihood doesn't imply sample quality

$$\log\left[0.01p(\mathbf{x}) + 0.99q(\mathbf{x})\right] \geq \log\left[0.01p(\mathbf{x})\right] = \log p(\mathbf{x}) - \log 100$$

Proportional to dimension d Constant

- With high dimension d, likelihood can be very large
  - 32*32*3 image -> $\mathrm{d} = 3072$, and $\log p(x)$ may be something like -3000

- What does this imply?
- ***The log likelihood of the mixture model is at worst –log100 worse than the ideal model, which is small compared to the actual log likelihood value scale***

Good sample doesn’t imply high likelihood

- A model that memorizes all training data
  - All generated samples are identical to training data
    - Best possible quality of samples
  - All test set data are assigned 0 probability
    - Lowest possible on likelihood

- **Likelihood and sample quality do not imply each other**
- **Likelihood is not everything. We still need to evaluate samples**

Evaluation paradigm

- Usually we have train set and test set
- After training on the train set, we apply the model to test set
  - Density evaluation
    - Are the models assigning test data high probability?
  - Sample evaluation
    - Are samples generated from the model good enough? (Or are they “close” enough to the distribution of test set data?)
  - Downstream tasks
    - Is the model supporting desired application well?

- **Evaluating GMs is multi-objective**

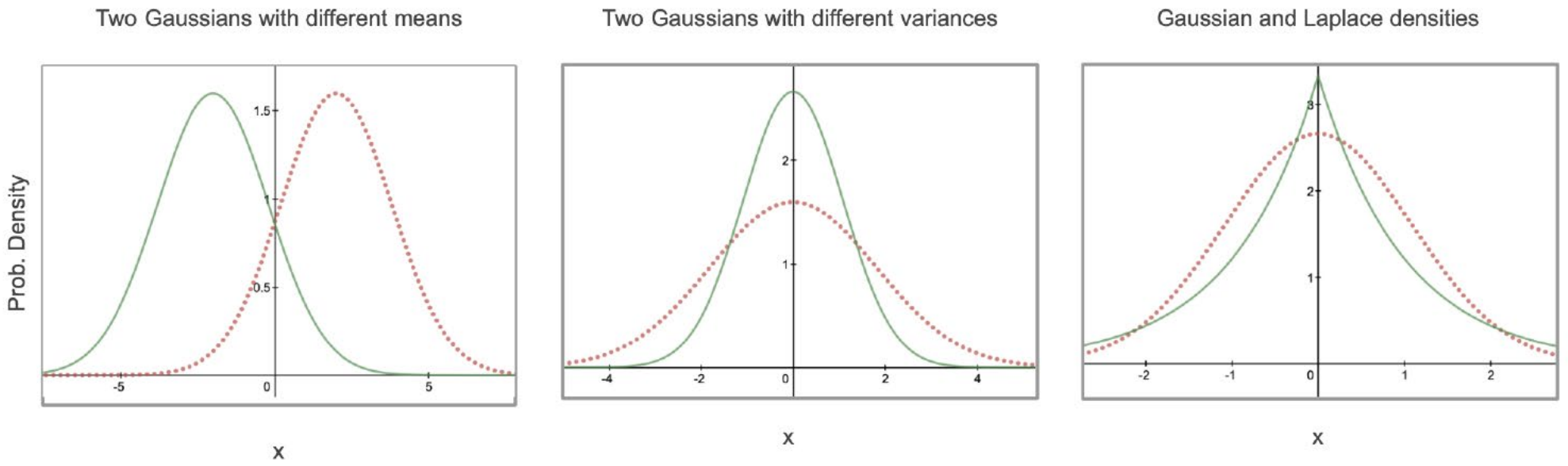
Evaluating sample quality

- Can we look at samples directly without the density function?
- Quantitative metrics
  - Inception Scores (IS)
  - Frechet Inception Distance (FID)
  - Maximum Mean Discrepancy (MMD)
    - Kernel Inception Distance (KID)
  - Precision & recall

Evaluating sample quality

- Assume we only have access to generated samples and test set
  - What can we do to evaluate the quality of samples?
- Consider the question: **given $\boldsymbol{S_1}$ drawn from distribution $\boldsymbol{P}$ and $\boldsymbol{S_2}$ drawn from $\boldsymbol{Q}$, are $\boldsymbol{P}$ and $\boldsymbol{Q}$ the same distribution?**
  - Null hypothesis H_0: $P = Q$
  - Alternate hypothesis $H_1 : P \neq Q$
- Design and test statistic T computed from S_1 and S_2
  - If $T < \alpha$ then accept H_0 else reject it
- Also called **two-sample test**

Two-sample test



- **Think**: what T should we design in these cases?

https://github.com/kuleshov-group/dgm-resources/blob/main/slides_pdf/lecture09.pdf

Two-sample test

- Two-sample test is likelihood free: only evaluate on samples
- In practice: usually assume P is data distribution and Q is model distribution
- Designing a good statistic T is not easy

- FID, MMD, and KID introduced below are based on the form of two-sample test

Inception score (IS)

- Assumptions:
  - We have labels for data;
  - Model distribution $p_\theta(x)$
  - We have a (trusted) classifier (usually pretrained) for data samples $c(y|x)$;
- **Sharpness**

$$S = \exp\left[\mathbb{E}_{x\sim p_\theta(x)} \int c(y|x)\log c(y|x)dy\right] = -\exp\left[\mathbb{E}_{x\sim p_\theta(x)}\left[\mathbb{H}(c(y|x))\right]\right]$$

- High sharpness:
  - Low entropy when the classifier is classifying generated samples
  - The classifier is more confident
  - Model is generating more distinguishable samples / easy to classify

Inception score (IS)

- Define classifier's marginal predictive distribution

$$c(y) = \mathbb{E}_{x \sim p_\theta(x)}[c(y|x)]$$

- **Diversity**

$$\begin{aligned} D &= \exp\left[-\mathbb{E}_{x \sim p_\theta(x)} \int c(y|x) \log c(y) dy\right] \\ &= \exp\left[-\int \mathbb{E}_{x \sim p_\theta(x)}[c(y|x)] \log c(y) dy\right] \\ &= \exp\left[-\int c(y) \log c(y) dy\right] = \exp\left[\mathbb{H}(c(y))\right] \end{aligned}$$



Low diversity



High diversity

- High diversity
  - Generated samples have high diversity
  - Distributed across all classes

https://github.com/kuleshov-group/dgm-resources/blob/main/slides_pdf/lecture14.pdf

Inception score (IS)

- Define IS

$$\mathrm{IS} = S \times D = \exp\left[\mathbb{H}(c(y)) - \mathbb{E}_{x\sim p_\theta(x)}\left[\mathbb{H}(c(y|x))\right]\right]$$

$$\log(\mathrm{IS}) = \mathbb{H}(c(y)) - \mathbb{E}_{x\sim p_\theta(x)}\left[\mathbb{H}(c(y|x))\right] = \mathbb{E}_{x\sim p_\theta(x)}\left[\mathrm{KL}[c(y|x)||c(y)]\right]$$

- Think: what if this is all data that the model generates?
  - What would happen to the S and D? And IS?



- Shortcomings
  - IS does not evaluate p(x) directly
  - Rely on classifier and needs the data to be classifiable
  - Does not account for diversity per class
    - If a model generates one perfect sample per class, the IS would be perfect

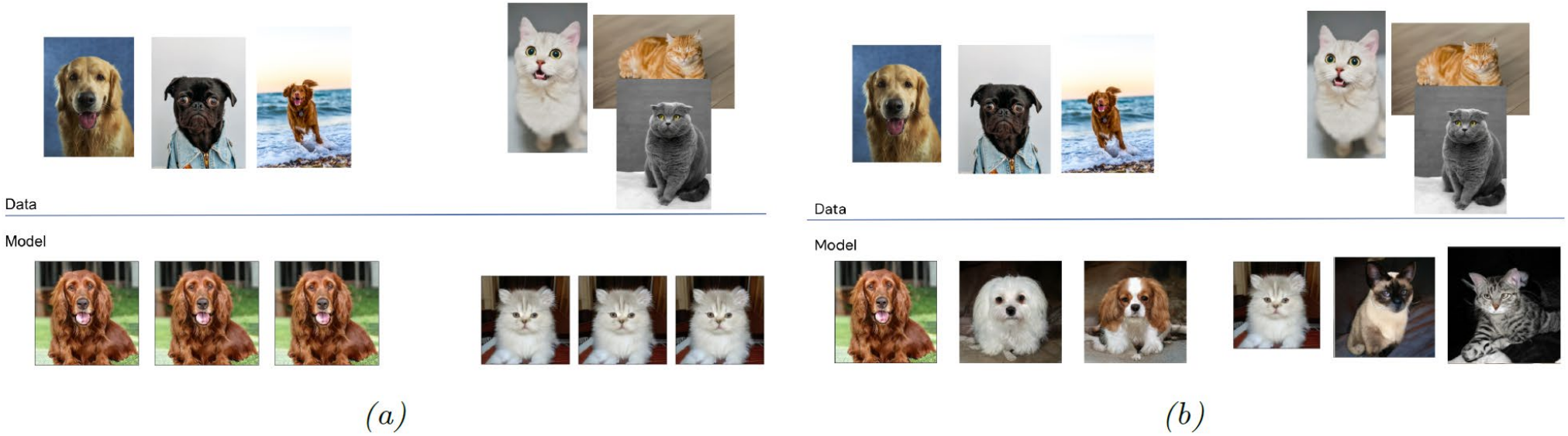
Frechet Inception Distance (FID)

- FID measures similarities in the **feature** representations, instead of class labels
  - Still requires the feature extractor
- Define sample set S and test set T
  - Get feature representations F_S and F_T respectively
- Fit two multivariate Gaussian models of each of them
  - Mean: μ_S and μ_T; covariance: Σ_S and Σ_T

$$\mathrm{FID} = ||\boldsymbol{\mu}_S - \boldsymbol{\mu}_T||^2 + \mathrm{Tr}\left[\Sigma_S + \Sigma_T - 2(\Sigma_S\Sigma_T)^{1/2}\right]$$

  - Low FID is better

IS vs. FID



- Think: how would the IS and FID be with cases a and b?
  - FID captures in-class diversity better than IS
  - However, FID is highly biased (highly influenced by the number of samples used)

Maximum Mean Discrepancy (MMD)

- Definition with respect to distribution P and Q:

Kernel mean embedding

$$\mathrm{MMD}(p,q) = ||\boldsymbol{\mu}_p - \boldsymbol{\mu}_q||_{\mathcal{H}_k}, \quad \boldsymbol{\mu}_p = \mathbb{E}_{\mathbf{x}\sim p}[\phi(\mathbf{x})], \quad \boldsymbol{\mu}_q = \mathbb{E}_{\mathbf{x}'\sim q}[\phi(\mathbf{x}')]$$

- Defined in RKHS, ϕ is feature representation (more math background not included)
- **Kernel Inception Distance (KID): the square of MMD**
  - With a suitable kernel function K

$$\mathrm{KID}(p,q) = \mathrm{MMD}^2(p,q) = \mathbb{E}_{x,x'\sim p}[K(x,x')] + \mathbb{E}_{x,x'\sim q}[K(x,x')] - 2\mathbb{E}_{x\sim p, x'\sim q}[K(x,x')]$$

$$= \frac{1}{N^2}\sum_{n=1}^{N}\sum_{n'=1}^{N} K(x_n, x_{n'}) + \frac{1}{M^2}\sum_{m=1}^{M}\sum_{m'=1}^{M} K(x'_m, x'_{m'}) - \frac{2}{NM}\sum_{n=1}^{N}\sum_{m=1}^{M} K(x_n, x'_m)$$

Kernel Inception Distance (KID)

- **Kernel Inception Distance (KID):**

$$\mathrm{KID}(p,q) = \mathrm{MMD}^2(p,q) = \mathbb{E}_{x,x'\sim p}[K(x,x')] + \mathbb{E}_{x,x'\sim q}[K(x,x')] - 2\mathbb{E}_{x\sim p,x'\sim q}[K(x,x')]$$
$$= \frac{1}{N^2}\sum_{n=1}^{N}\sum_{n'=1}^{N} K(x_n, x_{n'}) + \frac{1}{M^2}\sum_{m=1}^{M}\sum_{m'=1}^{M} K(x'_m, x'_{m'}) - \frac{2}{NM}\sum_{n=1}^{N}\sum_{m=1}^{M} K(x_n, x'_m)$$

- KID is comparing the similarity between samples within p and q individually to the samples from the mixture of p and q

Recall what a kernel function does: computing high dimensional dot products without explicit feature representation

Kernel Inception Distance (KID)

- Why is KID powerful?
  - No need to compute ϕ directly
  - Feature can be very high dimension and capture high order moments of the distributions
- Unbiased compared to FID

Precision & recall

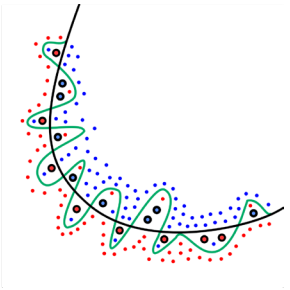
- We have implicitly mentioned some failure modes so far
- Single metric can’t tell all of them



Low fidelity:
unrealistic samples



Mode collapse: Model covers
only part of the data distribution



Memorization (overfitting):
reproduces training examples

Precision & recall

- Reference distribution P, model distribution Q
- **Precision**: how much generated data (Q) lies near the real data manifold (P)
- **Recall**: how much of the real data manifold (P) is covered by the generator (Q)

Definition 1. *For $\alpha, \beta \in (0, 1]$, the probability distribution Q has precision α at recall β w.r.t. P if there exist distributions μ, ν_P and ν_Q such that*

$$P = \beta\mu + (1 - \beta)\nu_P \quad \textit{and} \quad Q = \alpha\mu + (1 - \alpha)\nu_Q. \tag{3}$$

Sajjadi, Mehdi SM, et al. "Assessing generative models via precision and recall." Advances in neural information processing systems 31 (2018).

Precision & recall

Definition 1. *For $\alpha, \beta \in (0, 1]$, the probability distribution Q has precision α at recall β w.r.t. P if there exist distributions μ, ν_P and ν_Q such that*

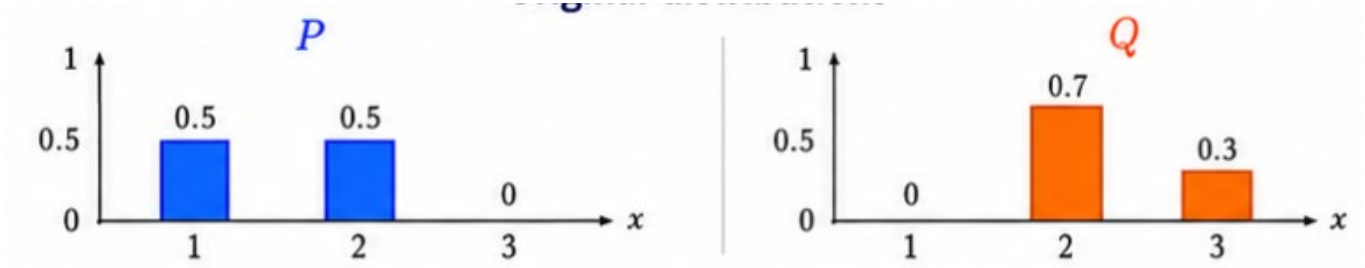
$$P = \beta\mu + (1 - \beta)\nu_P \quad \textit{and} \quad Q = \alpha\mu + (1 - \alpha)\nu_Q. \tag{3}$$

- μ describes the common part between P and Q
- υ_P and υ_Q describes the remaining “individual” parts

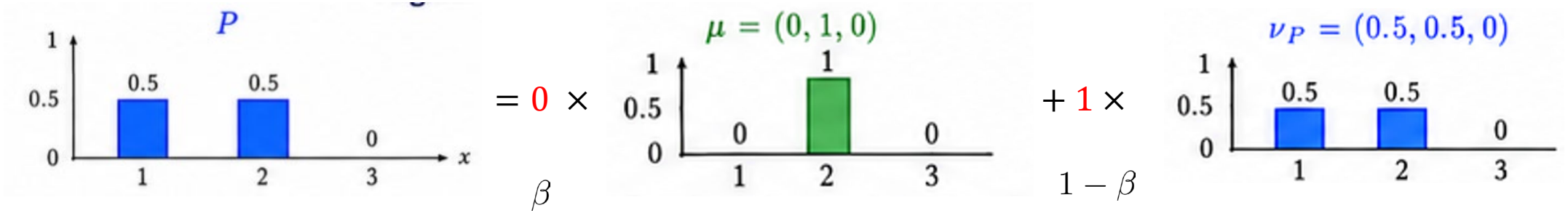
Precision & recall

$$P = \beta\mu + (1-\beta)\nu_P$$
$$Q = \alpha\mu + (1-\alpha)\nu_Q$$

- Suppose with distributions P and Q



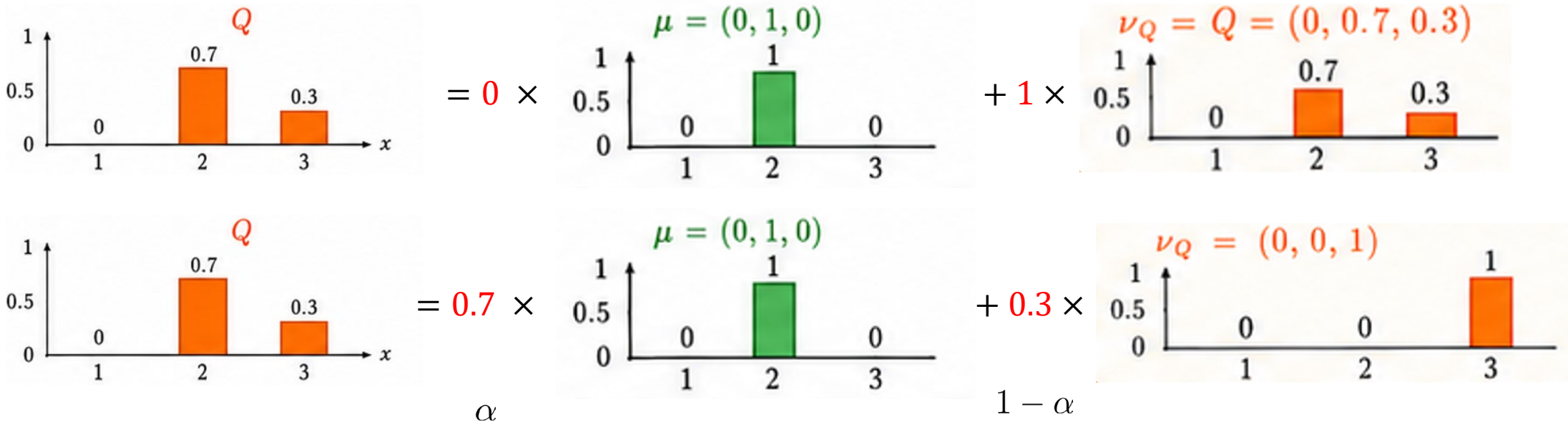
- Think: where is μ defined? (Hint: common part of P and Q)
  - μ is only defined on $\{2\}$ since that's the only common support
- **Think**: If we set $\beta = 0$, what values could α be?



Precision & recall

$$P = \beta\mu + (1-\beta)\nu_P$$
$$Q = \alpha\mu + (1-\alpha)\nu_Q$$

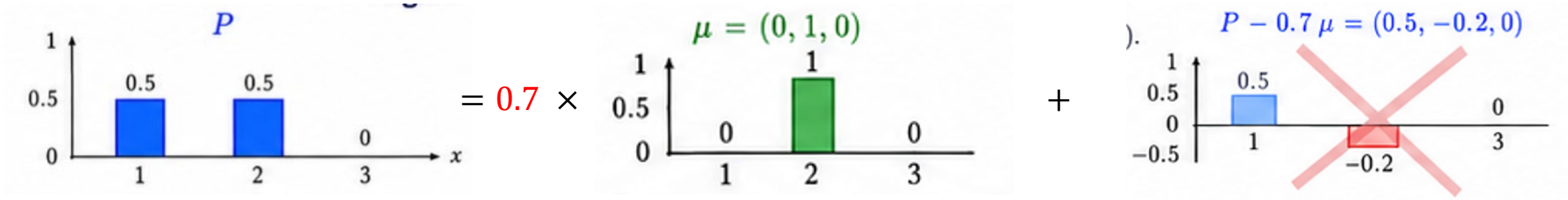
- α is not unique



- **Think**: what's the maximum value of α?
  - Answer: 0.7, since Q has weight 0.7 on common support $\{2\}$
  - What happens if we try to set $\alpha > 0.7$?

Precision & recall

- Similarly we can find other possible values of α for different βs
- What if we try to set $\beta > 0.5$?
  - Suppose at 0.7
  - We get negative mass



- **Only certain combinations of (α, β) are valid -> PRD**

Precision & recall

- PRD examples

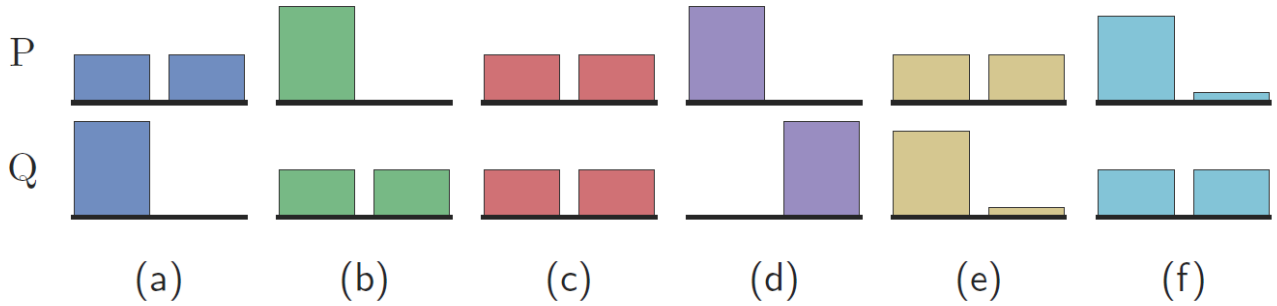


Figure 2: Intuitive examples of P and Q.

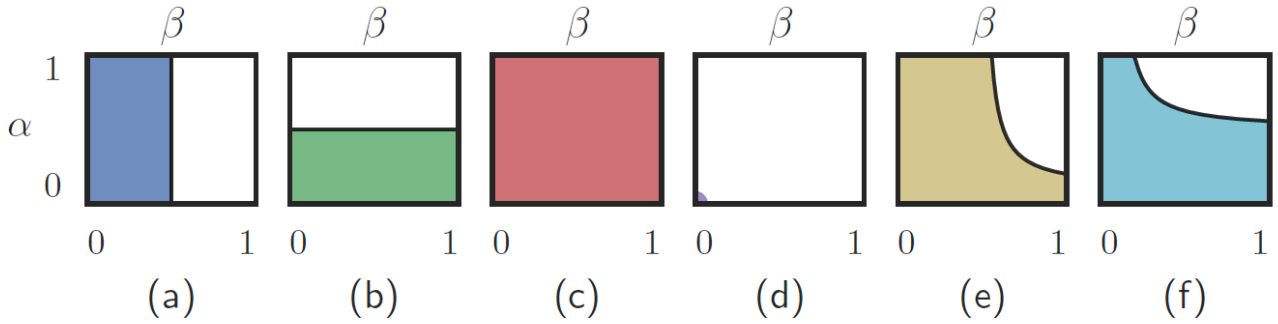


Figure 3: $\mathrm{PRD}(Q, P)$ for the examples above.

Sajjadi, Mehdi SM, et al. "Assessing generative models via precision and recall." Advances in neural information processing systems 31 (2018).

Assessing overfitting

- What if a model just memorizes all training data (overfitting)?
- **Think**: what happens to the features space when overfitting?
  - Generated samples are nearly identical around one sample
- Visual inspection: check K nearest samples in feature space from pretrained classifier





Brock, Andrew, Jeff Donahue, and Karen Simonyan. "Large scale GAN training for high fidelity natural image synthesis." arXiv preprint arXiv:1809.11096 (2018).

Drawbacks of pretrained classifiers

- Most metrics introduced so far require pretrained classifiers
- However, one might not have the classifier trained on identical dataset used for generation
- Classifier might not be reliable enough

Downstream tasks concerns

- What is the model intended for?
  - Image editing/denoising: reconstruction quality;
  - Text generation: human preference/truthfulness;
  - Data augmentation: training quality after augmentation

- **A model is good only when it performs well on the application that it's intended to support.**

Summary

- Evaluation by likelihood
  - Direct computation or approximation
- Evaluation by sampling
  - IS
  - FID
  - MMD & KID
  - Precision & recall
- Other concerns of evaluation
  - Recognizing overfitting
  - Drawbacks of lack of pretrained classifier

Summary

- Quantitative evaluation of generative models is challenging
- Metrics can significantly vary based on end goal: density estimation, sampling, latent representations
  - Know their advantages and drawbacks
- Evaluation should support downstream tasks / end goals, if there are

- ***There is no universal best metric. Evaluate the distributional property you care about.***

References and further reading

- Theis, Lucas, Aäron van den Oord, and Matthias Bethge. "A note on the evaluation of generative models." *arXiv preprint arXiv:1511.01844* (2015).
- Sajjadi, Mehdi SM, et al. "Assessing generative models via precision and recall." Advances in neural information processing systems 31 (2018).
- Murphy, Kevin P. Probabilistic machine learning: Advanced topics. MIT press, 2023.
- https://kuleshov-group.github.io/dgm-website/
- Brock, Andrew, Jeff Donahue, and Karen Simonyan. "Large scale GAN training for high fidelity natural image synthesis." arXiv preprint arXiv:1809.11096 (2018).